What this course is for

Five principles

Florian Gärtner

Where we are today

→	**What this course is for** ← you are here
	Getting it running
	Try it out
	The terminal, and how to read what Claude does
	Lunch
	A working day with Claude Code
	What the model reads, predicts and weighs
	A model, a menu, and a loop
	It doesn’t remember. It’s handed everything again.
	Wrap, done at 17:00

Research is already changing

Scientists who started drafting with LLMs produced 24 to 89% more papers, depending on field.

Kusumegi et al. 2025.

Two finance professors had Claude write hundreds of complete asset-pricing papers, each with its own theory for a return predictor their code had found. They published the result in the *Journal of Economic Literature*, as a warning.

Novy-Marx & Velikov 2026.

The AEA now runs an AI check on papers at the revise-and-resubmit stage. Since April 2026, the DFG lets reviewers use AI in a supporting role.

AEA 2026; DFG 2026.

Expected quality is an equilibrium

A paper is judged against the quality referees and readers expect. Authors aim for that quality, and what they deliver decides what is expected next.

- With Claude, more details can be right, such as a working replication package
- Claude also finds more small problems, in your work and everyone else’s
- So the expected quality rises
- A good paper from 2017 will probably look weak in 2027, though not a word of it has changed
- The same holds for your own Claude: a setup left alone falls behind

> **Keeping up gets harder without these tools**
>
> This course teaches you to keep improving your own Claude, and yourself.

Five principles

1. You will learn to use Claude Code, and to keep improving it.
2. On foundations that outlive it.
3. Inside a workflow, not a chat window.
4. As a coauthor. Which means learning to be one.
5. And a better scientist, with or without AI.

How you steer Claude decides the outcome

Anthropic studied about 400,000 Claude Code sessions from about 235,000 people, October 2025 to April 2026. People made about 70% of the planning decisions and 20% of the execution decisions. In sessions that hit trouble, 4% of novice-rated sessions still ended in a verified success, against 15% of expert-rated ones. In sessions that produced code, the ten largest occupations all came within seven points of software engineers.

Hitzig et al. 2026.

Claude does most of the typing. Deciding what to do, and rescuing a session that goes wrong, is your part. You can learn it, and it does not take a programmer.

1. You will learn to use Claude Code, and to keep improving it

By the end, Claude Code works on your own project. Not a toy example. Yours.

And Claude Code works the way you work, because you build it out yourself:

- Context engineering: what Claude knows when a session starts (instructions, memory, notes from the last session), and how to keep it lean
- Skills and custom commands for the jobs you repeat
- Connections to outside services (MCP servers), such as a literature database
- Permission rules and hooks that stop Claude before it does what you do not want

You start from a kit, not from nothing: this morning you get one with all of this set up, and the course opens each part when it gets to it, so you can make it yours.

How fast this moves

In early 2025, AI made 16 experienced developers 19% slower on 246 real tasks, though they believed it had made them 20% faster.

METR 2025.

A year later the same team could not repeat the study as designed, because more and more developers would not do half their work without AI, even when paid $50 an hour.

Becker et al. 2026.

Since 2023, the length of task the best models finish on their own has doubled about every four months.

METR 2026.

Any number about these tools dates within a year, the ones on these slides included. So this course spends its time on what stays true when the tool changes.

2. On foundations that outlive it

Something will replace Claude Code, probably soon. Almost everything here survives it.

How the tools work

- Your machine and its terminal
- Version control
- What an LLM does, and what it trades off
- How to swap in a different model

How to work with them

- Checking output before it reaches a paper
- Containing an assistant that acts as you
- Searching your own library with it
- Designing a pipeline of many steps

The concrete fixes this course teaches are different. They may be overtaken soon, and better ones may already exist that we have not met. Since September 2026, for example, Anthropic has been testing shared project memory and parallel threads in Claude Code's cloud sessions.

> **If Anthropic went bankrupt on Thursday**
>
> You would still be working on Friday.

Same task, different setup

On applied econometric tasks, a chatbot that writes one script succeeded 74% of the time. An agent that runs and revises its own code succeeded 96% of the time, at about eight cents more per run. With the agent, worked examples in the prompt helped far less than they helped the chatbot, and the gaps between Stata, R and Python largely disappeared.

Galiani, López & Sosa 2026.

So this course spends little time on how to word a request and much on how the work is set up.

3. Inside a workflow, not a chat window

The question is not what one prompt can do. It is what your project has to look like, so that Claude's help does not become a mess you cannot audit.

In this course, that means:

- A project layout Claude can find its way around
- Version control, so every change can be traced and undone
- A pipeline that rebuilds every table from the raw data

Who gains from it

"AI won't take your job – it's someone using AI who will take your job."

Richard Baldwin, 2023.

Which somebody? 133 patent lawyers at eleven US law firms got an AI drafting assistant for three months, or not. With it, their drafts improved, the juniors' most. Then everyone reviewed a patent application without AI. Treated seniors beat the controls, while treated juniors did not on average. In the authors' words: "The largest gains from AI thus accrued to the lawyers who retained the least."

Autor et al. 2026.

Early in a career, Claude can make the work better while you learn less from doing it.

4. As a coauthor

Claude is a coauthor, even though you cannot name Claude as one. Authorship requires accountability, and the accountability is entirely yours. So your job changes: less producing, more deciding, checking, and being answerable for it.

Pass Claude's output on unread, and you become a "meat proxy".

Gruhn 2026.

In the words of a Princeton series on Claude Code for applied economists, "Verification is the new bottleneck."

Goldsmith-Pinkham & Markus' Academy 2026.

Your coauthors have a Claude too. Handover files, commit messages, branches, the same project conventions: each comes later in the course, and each is written so that your Claude and theirs agree on what happened. Neither of you has to explain the project to the other.

Rerunning published research

Independent researchers reran 110 articles from leading economics and political science journals, all of which require authors to share their data and code. More than 85% of the published claims came out the same. About a quarter of the studies contained coding errors, a figure the authors call likely to underestimate the true prevalence.

Brodeur et al. 2026.

Errors like these are found when someone else can rerun your code.

5. And a better scientist, with or without AI

Much of what this course teaches is good research practice in its own right:

- Knowing your own machine: files, paths, the terminal
- Version control with git, and a copy of the project on GitHub
- A project laid out so that someone else can rerun it
- Every number in the paper produced by a script

Good scientists should do all of this whether or not they use AI.

What we learned from daily use

What Claude reads

- Which instructions Claude should read at every start, and which only when needed
- How to describe a skill so that Claude uses it when it should

Keeping a session on track

- When to rewind a session instead of correcting it
- How to tell Claude in advance when a task is finished

Knowing the result is right

- How to test whether a check would catch a real failure
- Why a second session should review the first one’s work

Work that runs without you

- How to run two sessions on one project without them mixing up each other’s work
- How to build a literature search that runs by itself, and when it should stop